

The Story of CSC 494

AI-Driven IoT System Development

Even though this is labeled as "CSC" and 494, this course is strongly connected to ASE curriculum.

- This course is project based course similar to other ASE courses.
- This course uses the same tools and practices as other ASE courses.
- We focus on problem solving using tools and rules.

The problem domain happens to be IoT (Embedded Systems) and the tools happen to be AI.

Being on the Same Page

Unfortunately, this is **Not (or cannot be)**

- Fun with IoT course.
 - We don't have time for that.
- Advanced IoT course.
 - What's the meaning of "advanced" anyway in the age of AI.
 - AI knows everything about 'How' and 'What'.

- Arduino Programming course.
 - You will be a master of arduino programming after this course, but this is a side effect.
- ESP32 course.
 - You will understand how to use ESP32 for BLE/Wifi applications, but this is also a side effect.

This Course does **Not Teach you (No course can)**

- How to use AI tools X, Y, or Z
 - What's the meaning of that when things are dramatically changing almost every month and year.
- How to make good ChatGPT prompts.
 - What's the meaning of that when AI can make the best prompts for you.

- Or, how to ... AI ...
 - Whatever you learn today can/will be obsolete tomorrow (at least next year).

But, from experience, we know that we will figure out anything when we learn by **solving real world problems**.

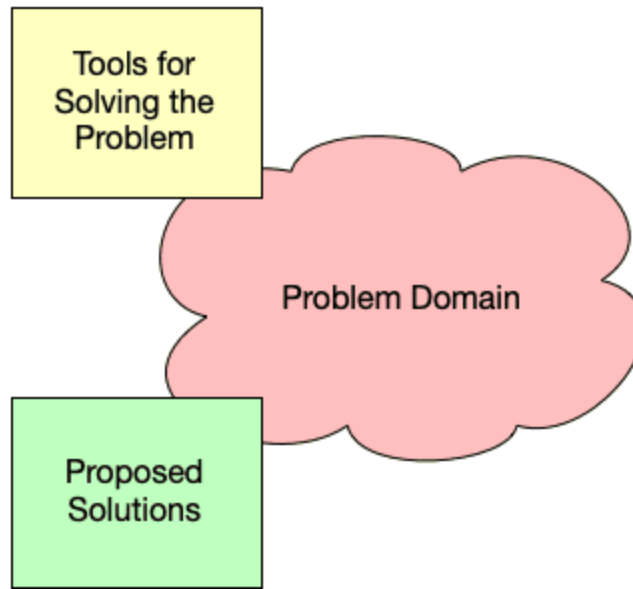
- This is still the best (possibly only way) to learn something meaningful in the age of AI.

This Course is About

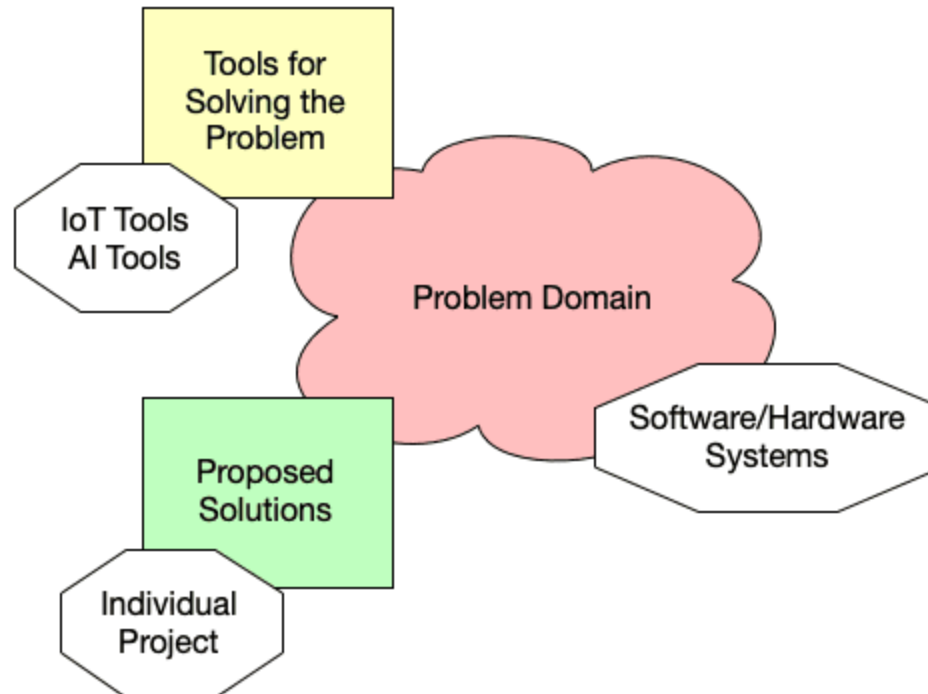
- **Thinking** and **learning** how to solve IoT domain problems.
- **Using** the AI/IoT tools effectively to help you solve the problems.
- **Building** software/hardware system using Arduino/ESP32 platform to solve the problems.

In other words,

This is a problem-solving course using appropriate tools and building apps to prove that you can solve the given problems.



- The problem domain: Software/Hardware systems
- Tools: AI + IoT Tools
- Solutions: **You propose your solutions using the individual project**



So, run, before it's too late!

Or make the decision to **play the game**:

- how to solve problems effectively in the software/hardware systems (or embedded systems).
- how to use AI/IoT tools that know everything about 'What' and 'How' to solve problems in any domain

effectively and without shooting your feet.

How do we play this game?

- We will discuss any ideas in the domain of IoT (Embedded Systems) from the problem-solving perspective.
- We will discuss how to use AI tools as your second brain, or co-pilot, to solve problems effectively from the problem-solving perspective.
- We will build software/hardware systems using Arduino/ESP32 platform to solve the problems.

Understanding the ASE Course Context

Read the following before the next class to be on the same page (Part of HW1).

- Students should know how to use three core tools.
- (For Windows Users) Students should know how to use WSL2.
- Students should understand Software Engineering and ASE courses structure.

- Students should understand the ASE Team and Individual Projects.
 - There is no team project in this course.
 - However, the process & ceremonise of team project is used in individual project.
- ASE courses are about problem solving using tools/rules, and this is not exception.
 - We use IoT (Embedded Systems) as the application domain.
 - We use Ai tools to help us solve problems effectively.

Converted PDF Files:

- VSCode:

https://github.com/nkuase/ASE/blob/main/ase_onboard/1_vsc.pdf

- Markdown/Marp:

https://github.com/nkuase/ASE/blob/main/ase_onboard/2_marp.pdf

- GitHub:

https://github.com/nkuase/ASE/blob/main/ase_onboard/3_github.pdf

For Windows Users

As a software engineering student, you will need to use Windows Subsystem for Linux (WSL) in ASE courses.

- Marp:

https://github.com/nkuase/ASE/blob/main/ase_onboard/4_wsl2_linux_on_windows.md

- PDF:

https://github.com/nkuase/ASE/blob/main/ase_onboard/4_wsl2_linux_on_windows.pdf

Understanding Software Engineering & ASE Courses

(https://github.com/nkuase/ASE/tree/main/ASE_story/SE)

- SE and ASE Courses:

https://github.com/nkuase/ASE/blob/main/ASE_story/SE/se_and_ase_courses.pdf

Understanding ASE Team & Individual Projects

(https://github.com/nkuase/ASE/tree/main/ASE_story/project)

- Project Theory:
https://github.com/nkuase/ASE/blob/main/ASE_story/project/project_theory.pdf
- Team Project Practice:
https://github.com/nkuase/ASE/blob/main/ASE_story/project/team_project_practice.pdf
- Individual Project Practice:
https://github.com/nkuase/ASE/blob/main/ASE_story/project/individual_project_practice.pdf

Two Information Sources

GitHub Repositories

- NKUASE/ASE (ASE Repo): General ASE Course Materials
 - <https://github.com/nkuase/ASE>
- NKUASE/csc494 (CSC494 Repo): CSC494 Specific Materials
 - <https://github.com/nkuase/csc494>

Canvas Page

- Schedules
- Announcements
- Assignment Submissions
- Grades

Why Two Sources?

1. GitHub Repository

- Artifacts of the course
 - Documents: Slides, Project Prototypes, Code, and other files
- Students can access the materials before the course starts and after it ends

2. Canvas Page

- Course Management
 - Schedules, Announcements, Assignment Submissions, Exams, and Grades
 - Students need to log in to access the course materials
 - No access before the course starts and after it ends

The Two Focuses of ASE 285

Understanding Embedded Systems

- Software Engineers need to understand hardware to be the best problem solvers.
- We have more and more integrated software/hardware systems in our daily lives.
- Embedded Systems are everywhere, so understanding embedded systems can open new opportunities.

Understanding AI Tools

- AI is transforming the way we work, learn, and live.
- AI is changing the software engineering landscape dramatically.
- Software Engineers need to understand AI tools to be the best problem solvers.

Issues of Embedded Systems & AI Tools

1. Embedded Systems

- There are many concepts and technologies to understand.
- There are many hardware and software platforms to choose from.
- There are many tools to use for development and debugging.

2. AI Tools

- AI tools and methods are changing rapidly, so it's practically meaningless to learn any specific tools.
- When we delegate problem solving to AI tools, we should ask "Why do companies hire me if AI can do my job?"
- If we cannot understand what AI tools are doing, or the results from AI tools, we are in deep trouble.

CSC 494 Approach to the Issues

1. Embedded Systems

- We focus on project; understand the problem domain and try to solve the problems by focusing on building software/hardware systems.
- We learn any necessary concepts and technologies as needed for the projects as we go.
- **We take advantage of AI tools to help us learn and build the projects effectively.**

2. AI Tools

- We consider AI tools as our personal tutors, assistants, pair programmers, and coaches.
- We **never** delegate our problem solving to AI tools; we solve problems **with** AI tools.
- We **always** understand what AI tools are doing, and the results from AI tools.

In short:

- We use AI as tools to understand how to build software/hardware systems to solve problems effectively.
- We understand how we can survive and prosper in the age of AI as software engineers and problem solvers.
- We build the software/hardware systems to prove that we can solve the problems effectively.

Software Engineering & Software Engineers

1. Students learn how to build "**High-Quality Software Product**".
 - High Quality Software means the software that is effectively and easily *maintained* and *evolved* over time.
 - Software Product means the software that people are paying money to use.
 - Professionals make money by building High-Quality Software Products.

2. Students learn how to "**Solve problems**" by "**Managing complexity**".

- Software Engineering is about managing complexity.
- Software Engineering is about problem solving using rules and tools.
- So, software engineers are **professionals** who **solve problems** by **managing complexity** using **rules** and **tools** in a **team**.

Software Engineering

- Software Engineering is about (1) rules and (2) tools to (3) build applications (Solve problems) in a (4) team.
 - We solve problems in the form of building applications.
- The name of Software Engineering game is **Managing Complexity**.
 - Most software engineering problems are from complexity from various sources, noticeably from changes.

Software Engineers

- We are professional **problem solvers**.
 - We **build high-quality** software products that people are willing to pay to use.
 - We **use rules and tools** to manage complexity in a team environment.

Software Engineering is Not

- Programming or Coding
 - Programming or Coding is just a small part of Software Engineering.
- Hacking or Quick and Dirty Solutions
 - Hacking or Quick and Dirty Solutions create more problems in the long run.
- Working alone
 - Software Engineering is a team sport.

Software Engineering is

- Making clients happy by delivering high-quality software products.
 - Keeping the promise we made to clients.
 - Making clients successful by solving their problems using the software we build.
 - Making clients come back by being reliable and trustworthy.

- Improving ourselves continuously to be better problem solvers.
 - Learning new tools and rules to manage complexity better.
 - Learning from our mistakes and successes.

In short, being a true professional problem solver to make our clients trust us and pay us to use.

- Think about any software engineering company that does not follow these principles. Maybe None!

You are a Problem Solver

- As long as you solve problems, you are a software engineer.
- This is the course to help you become a better software engineer by understanding how to solve problems effectively using AI and IoT tools.